



ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT073-3-2

Computer Systems Low Level Techniques

APU2F2506CS(DF,AI), APD2F2506CS(DF,AI)

HAND OUT DATE: 19 DECEMBER 2025

HAND IN DATE: 16 MARCH 2026

WEIGHTAGE: 60%

NAME: WONG JIN JIE

INTAKE CODE: APU2F2506CS

SUBJECT: CT073-3-2 CSLLT

PROJECT TITLE: DAILY CALORIE BUDGET MANAGER

INSTRUCTIONS TO CANDIDATES:

- 1 Assignment is to be submitted through online submission (Moodle)**
- 2 Students are advised to underpin their answers with the use of references (cited using the APA Referencing).**
- 3 Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
- 4 Cases of plagiarism will be penalized.**
- 5 Where the assignment should be submitted online (Moodle), the link to the source code and other relevant sources (where appropriate) should be attached together with the submission.**

Contents

1.0 Introduction on Assembly Language	4
2.0 Research and Analysis	5
2.1 Assembly in Malware Analysis	5
2.2 Assembly in Reverse Engineering	5
2.3 Memory Forensics and Buffer Overflows	5
3.0 Reverse Engineering	6
3.1 Definition of Reverse Engineering	6
3.2 Advantages of Reverse Engineering	6
3.3 Practical Analysis using IDA	7
3.4 Justification: Why Choose Reverse Engineering?	8
4.0 System Design using flowchart	9
5.0 System Screenshot	10
5.1 System Initialization and Goal Setting	10
5.2 Main Menu Interface	11
5.3 Standard Calorie Addition	12
5.4 Advanced Feature: Custom Input	13
5.5 Warning Mechanism and Negative Balance	14
5.6 Program Termination	15
6.0 Source Code with Explanation	16
6.1 Utility Functions (functions.asm)	16
6.1.1 Integer to String Conversion (iprint)	16
6.1.2 ASCII to Integer Conversion (atoi)	18
6.1.3 System Call Wrappers (slen, sprint, quit)	19
6.2 Main Program Initialization (main.asm)	20
6.2.1 Data and Memory Allocation	20
6.2.2 Program Entry and Goal Setting	21
6.3 The Main Application Loop (main.asm)	22
6.3.1 Status Display and Signed Arithmetic Logic	22
6.3.2 Warning Logic	23
6.4 Menu Selection and Execution(main.asm)	24
6.4.1 Menu Display and Input	24
6.4.2 Jump Table Implementation	25
6.4.3 Feature Execution (Standard vs. Custom)	26
6.4.4 Exit Program	27
7.0 Conclusion	28
8.0 References	29

Appendices.....	30
Complete Source Code	30
main.asm	30
functions.asm	32

1.0 Introduction on Assembly Language

Assembly language is a low-level programming language that allows human to utilize Central Processing Unit (CPU) through assembler by using specific instructions (Irvine, 2020). Unlike high-level programming languages such as Python or Java, assembly language is very similar to machine code, the assembler acts as the translator between human instructions and CPU (Stallings, 2022). The dependency on computer architecture makes assembly language programming unique, allowing human to straightaway work with computer registers and memory addresses. Assembly language programming is essential to provide fundamental understanding of computer architecture and helps on system optimization and debugging.

For this assignment, a "Daily Calorie Budget Management System" was developed using the x86 assembly language with the Netwide Assembler (NASM) on a Linux platform (Ubuntu). This project is mainly aiming to show the wide application of low-level programming. The system is a command-line interface (CLI) application which allows users to enter a daily calorie target, choose food options from a predefined menu, or simply type in the custom calorie amounts of their meals. The development is primarily focuses on the basic instructions, including arithmetic operations for budget calculations, loops for menu iteration, and conditional jumps for input error checking and warnings.

2.0 Research and Analysis

2.1 Assembly in Malware Analysis

In the domain of cybersecurity, malware analysis experts do not generally have access to the source code of malware. Rather, they generally receive compiled versions of malware. According to Attari (2023), to effectively perform malware analysis, it is important for cybersecurity professionals to be proficient in assembly language. This is because they can effectively use disassemblers to convert machine code into assembly language. This low-level analysis of malware will help cybersecurity professionals effectively identify the exact behavior of malware. For instance, they will be able to identify which files are being encrypted by malware or which servers malware is communicating with.

2.2 Assembly in Reverse Engineering

Reverse engineering is the process of "disassembling" the software to understand the internal architecture. EC-Council (2022) states that the security researcher needs to have a "good grasp" of assembly language to be able to do this. This is because, according to the EC-Council, "assembly language is the closest to the hardware that we can get. It is possible to uncover hidden vulnerabilities, proprietary protocols, or backdoors that are not visible on the surface level of the software. This is the key to 'vulnerability hunting,' which is the process of finding and fixing the vulnerabilities before the hackers do."

2.3 Memory Forensics and Buffer Overflows

Assembly language is also the basis for understanding memory attacks, specifically in buffer overflow attacks. According to Cobalt (2023), buffer overflow attacks happen when a program tries to write more data into a buffer than it can actually hold, which will cause to overwrite the memory that is adjacent to it. In order to analyze or prevent such attacks, it is necessary to have a sufficient understanding of how the CPU stack works at the register level. This is because the attacker usually attempts to utilize the EIP (Extended Instruction Pointer) register to redirect the program execution flow to the attacker's own assembly code, which is also known as shellcode. Therefore, it is necessary to have knowledge of assembly instructions such as MOV and JMP to perform memory forensics and exploit analysis.

3.0 Reverse Engineering

According to the above research, one of the most significant applications of assembly language is reverse engineering. This part also gives a breakdown of its concepts, benefits and even a practical demonstration using IDA Pro.

3.1 Definition of Reverse Engineering

Reverse engineering concerns the act of examining compiled code (e.g. an executable .exe file or an object file) to comprehend its design, architecture and logic. The original high-level source code (such as C++ or Java) is often not made public, and therefore, to convert the raw machine code into readable assembly code special tools such as disassemblers are used by the analysts. This will enable researchers to know precisely how the software works in the hardware level.

3.2 Advantages of Reverse Engineering

Reverse engineering has a number of benefits in technology industry:

1. **Malware Analysis:** It enables cybersecurity analysts to break down computer viruses and learn about their malicious without implementing the viruses.
2. **Vulnerability Discovery:** It is used by security researchers to discover concealed bugs or security vulnerabilities within commercial software to fix them before being exploited by hackers.
3. **Software Interoperability:** It assists the software developer to create software that allows them to communicate with undocumented systems or with third party applications by learning about their internal protocols.
4. **Recovering Lost Source Code:** In cases where the original source code of an old system has been lost accidentally by the company, reverse engineering can be used to reconstruct the logic of the compiled executable.
5. **Educational Uses:** It is a very useful educational resource about computer science students and computer programmers to learn about the process of constructing complex systems and compilers.

3.3 Practical Analysis using IDA

```

IDA View-A
Hex View-1
Local Types

.data:0000000000000000 ;
.data:0000000000000000 ; +-----+
.data:0000000000000000 ; | This file was generated by The Interactive Disassembler (IDA) |
.data:0000000000000000 ; | Copyright (c) 2026 Hex-Rays, <support@hex-rays.com> |
.data:0000000000000000 ; | Freeware version |
.data:0000000000000000 ; +-----+
.data:0000000000000000 ;
.data:0000000000000000 ; Input SHA256 : F7FFD2464004442BC1559A3B0A47251391383D7292FD2EFFA22C693BEFBEB
.data:0000000000000000 ; Input MD5 : A932566410821F446EBCC646ABD5DA1D
.data:0000000000000000 ; Input CRC32 : 257BD7F8
.data:0000000000000000 ; Compiler : GNU C++
.data:0000000000000000 ;
.data:0000000000000000 ; File Name : \\wsl.localhost\Ubuntu\home\user\char.o
.data:0000000000000000 ; Format : ELF64 for x86-64 (Relocatable)
.data:0000000000000000 ;
.data:0000000000000000 ; Source File : 'char.asm'
.data:0000000000000000 ;
.data:0000000000000000 .686p
.data:0000000000000000 .mmx
.data:0000000000000000 .model flat
.data:0000000000000000 .intel_syntax noprefix
.data:0000000000000000 ;
.data:0000000000000000 ; =====
.data:0000000000000000 ; Segment type: Pure data
.data:0000000000000000 ; Segment permissions: Read/Write
.data:0000000000000000 _data segment dword public 'DATA' use64
.data:0000000000000000 assume cs:_data
v .data:0000000000000000 msg db 41h ; A ; DATA XREF: .text:00000000000001A1o
.data:0000000000000001 db 0Ah
.data:0000000000000001 _data ends
.data:0000000000000001
.text:0000000000000010 ;
.text:0000000000000010 ; =====
.text:0000000000000010 ; Segment type: Pure code
.text:0000000000000010 ; Segment permissions: Read/Execute
.text:0000000000000010 _text segment para public 'CODE' use64
.text:0000000000000010 assume cs:_text
.text:0000000000000010 ;org 10h
.text:0000000000000010 assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing
.text:0000000000000010
.text:0000000000000010 public _start
.text:0000000000000010 _start:
v .text:0000000000000010 mov eax, 1
.text:0000000000000015 mov edi, 1
.text:000000000000001A mov rsi, offset msg
.text:0000000000000024 mov edx, 2
.text:0000000000000029 syscall ; LINUX - sys_write
.text:000000000000002B mov eax, 3Ch ; '<'
.text:0000000000000030 xor rdi, rdi
.text:0000000000000033 syscall ; LINUX - sys_exit
.text:0000000000000033 _text ends
.text:0000000000000033
.text:0000000000000033 end

```

Figure 3.3:IDA View

In order to illustrate the reverse engineering activity, a small object file (char.o) was examined under the IDA Pro. This program is intended to display one character, the letter A and a newline to the screen.

The data section of the program in the IDA View is shown in Figure 3.3. The disassembler was succeed to identify one variable called msg. The data is represented in the form of 41h (the A character in hexadecimal ASCII) and 0Ah (the newline character). This is the demonstration of how the data is stored in the memory before program execution.

The text segment, which carries the executable instructions, is also displayed in Figure 3.3. The assembly code makes a system call (syscall). It loads 1 to eax (syswrite), sets the destination of the output, loads the address of the message and sets the length at 2 bytes. Lastly, it prepares 3Ch (sysexit) to exit the program cleanly. This consideration recreates the flow of logic of the object file.

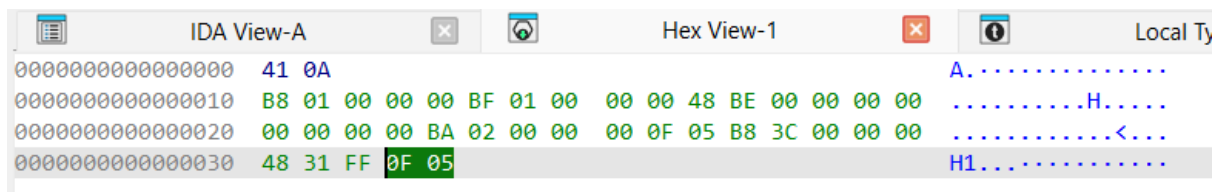


Figure 3.4: Hex View

Figure 3.4 indicates the Hex View alongside the IDA View. The Hex View displays the raw hexadecimal bytes that the CPU actually reads (e.g., 41 0A on the first line). Although human developers find the Hex View hard to read, the IDA View converts such raw bytes into the assembly instructions easily understood by humans in the Figure 3.3.

3.4 Justification: Why Choose Reverse Engineering?

The reason why reverse engineering was selected in this research section is that it is the final step between theory and practicality of learning the assembly language. Learning how to read and analyse compiled binaries is an elementary requirement as a student of computer science studying low-level techniques. It shows that assembly language is not only used to construct systems, but also to audit, secure, and comprehend them, which is an extremely demanded skill in the current cybersecurity environment.

4.0 System Design using flowchart

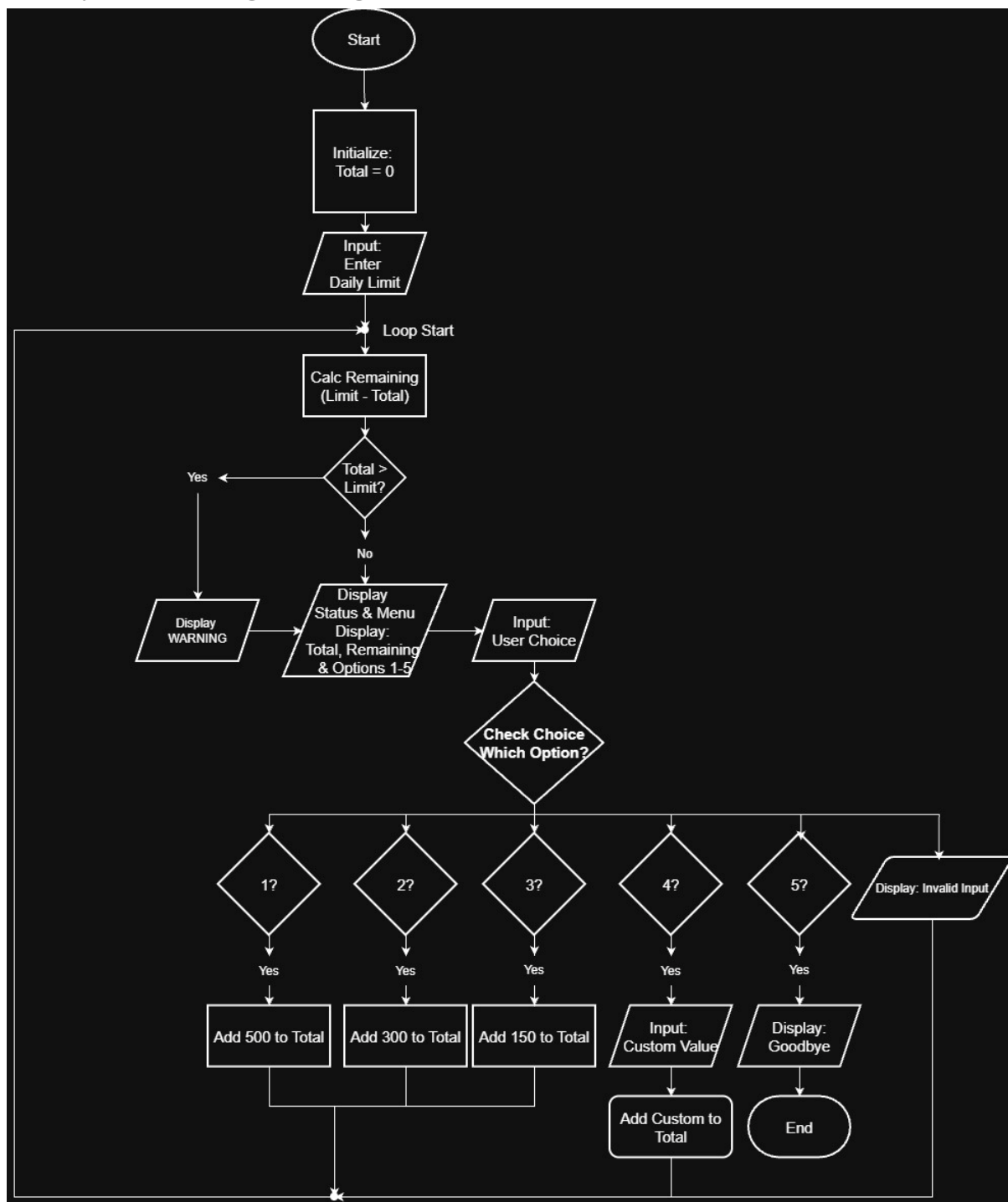
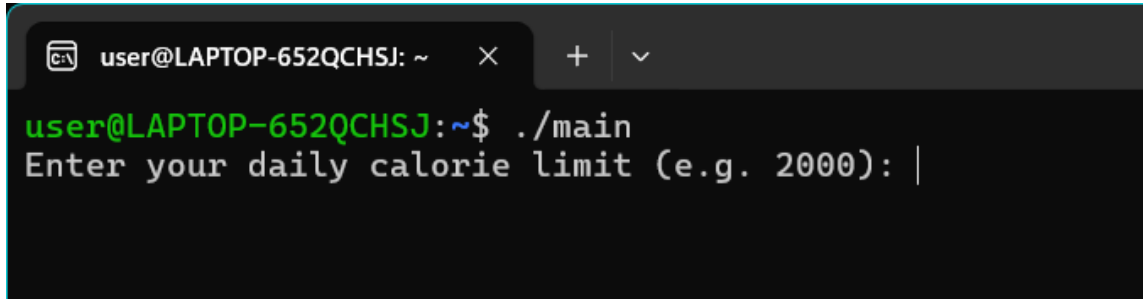


Figure 4.1: System Flowchart illustrating the main loop, input validation, and calorie calculation logic.

5.0 System Screenshot

This section acts as an user manual, which fully explains the operational flow of the Daily Calorie Budget Management System. It demonstrates the complete function of the application, from initialization of daily calorie budget setting, calculations and error handling based on user input, and termination.

5.1 System Initialization and Goal Setting

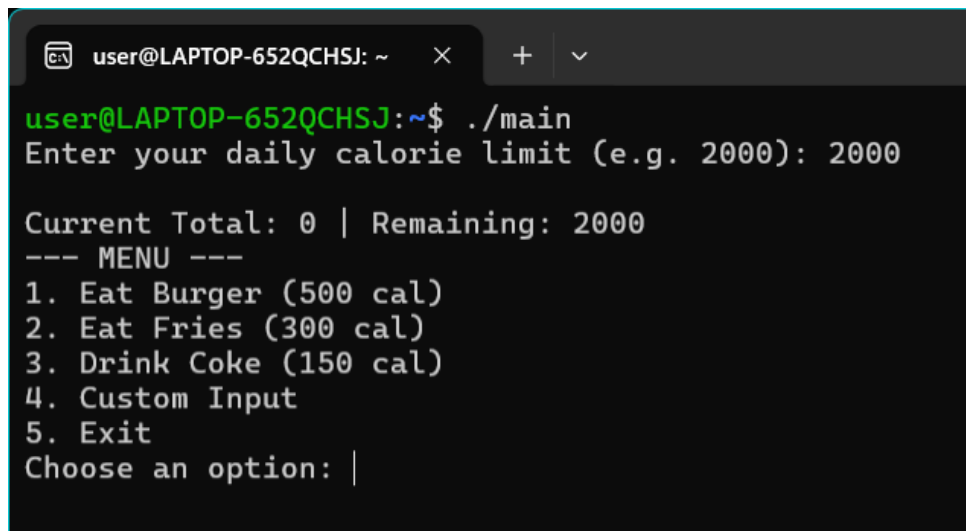
A terminal window with a dark background. The title bar shows 'user@LAPTOP-652QCHSJ: ~' and window control buttons. The terminal text shows a green prompt 'user@LAPTOP-652QCHSJ:~\$' followed by the command './main'. Below this, a white prompt 'Enter your daily calorie limit (e.g. 2000):' is followed by a vertical cursor line '|'.

```
user@LAPTOP-652QCHSJ: ~  
user@LAPTOP-652QCHSJ:~$ ./main  
Enter your daily calorie limit (e.g. 2000): |
```

Figure 5.1: System Initialization and Calorie Goal Input

While executing the program, the system first initializes all memory variables to zero. It will prompt users to insert a daily calorie budget as an initialized value of daily limit calorie. This input will be processed by using the ATOI (ASCII to Integer) function which is defined in functions.asm and stored as the baseline value for all calculations later.

5.2 Main Menu Interface

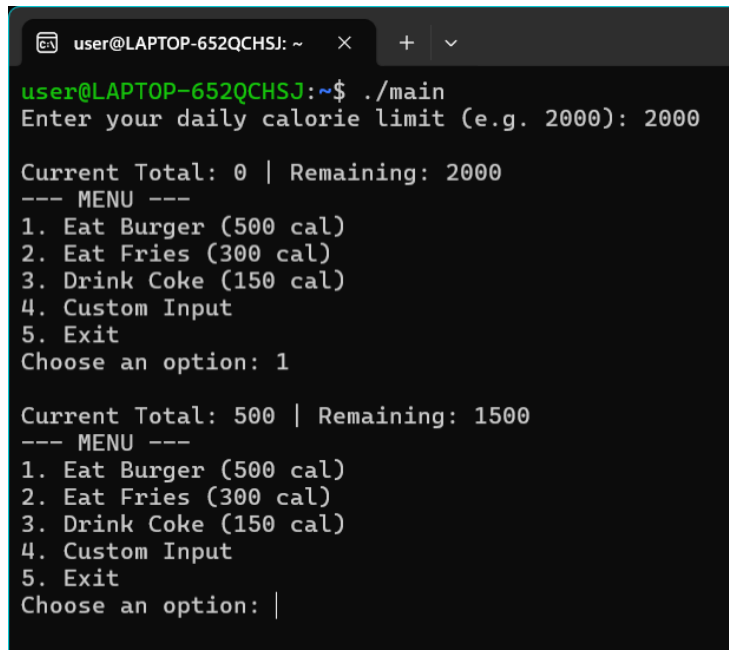


```
user@LAPTOP-652QCHSJ: ~  
user@LAPTOP-652QCHSJ:~$ ./main  
Enter your daily calorie limit (e.g. 2000): 2000  
  
Current Total: 0 | Remaining: 2000  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: |
```

Figure 5.2: Main Menu Interface displaying real-time status

After user inserts the target value, the system enters to the main loop and shows the user interface. The top portion of the interface shows the "Current Total" amount of calories and the "Remaining" amount. The menu contains five options: three standard food choices, one user custom input option, and an exit option.

5.3 Standard Calorie Addition

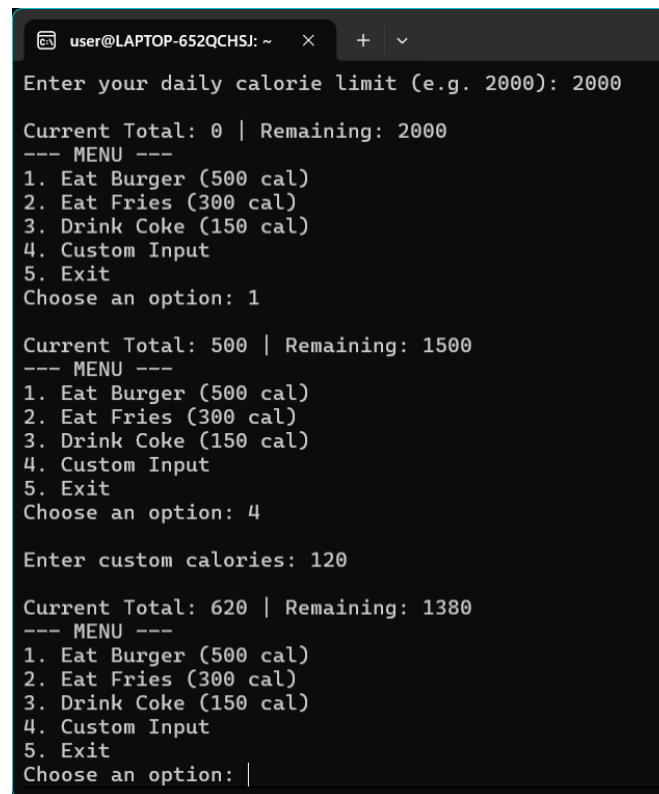


```
user@LAPTOP-652QCHSJ: ~  
user@LAPTOP-652QCHSJ:~$ ./main  
Enter your daily calorie limit (e.g. 2000): 2000  
  
Current Total: 0 | Remaining: 2000  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 1  
  
Current Total: 500 | Remaining: 1500  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: |
```

Figure 5.3: Standard Item Selection and Calorie Calculation

It also includes preset options for commonly consumed food items to ease the process of data entry. As shown in Figure 4.3 above, when Option 1, which represents Burger is chosen, an arithmetic operation that performs addition by 500 calories is carried out on the total. The interface is updated immediately to show the new total and the reduced balance.

5.4 Advanced Feature: Custom Input

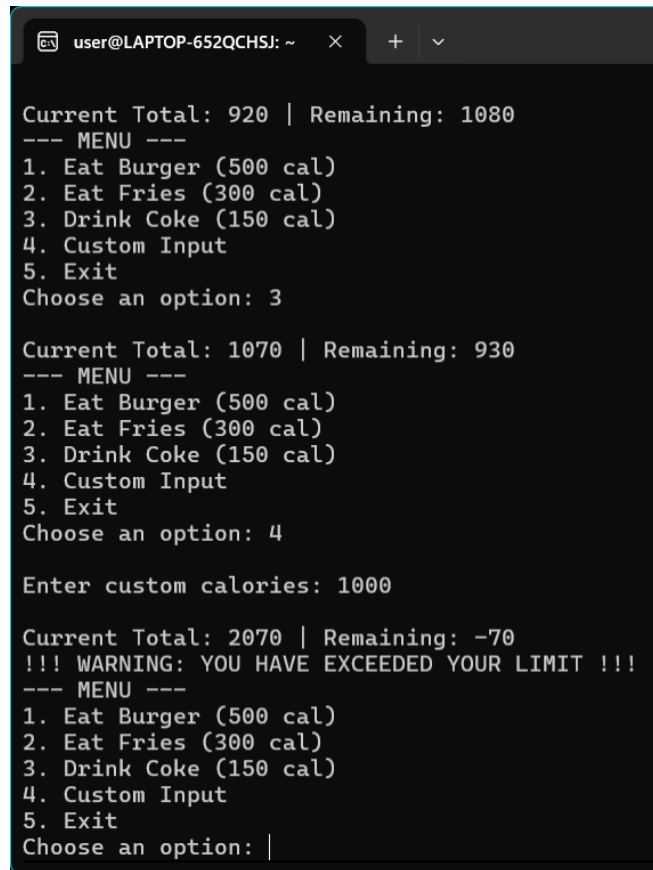


```
user@LAPTOP-652QCHSJ: ~  
Enter your daily calorie limit (e.g. 2000): 2000  
Current Total: 0 | Remaining: 2000  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 1  
  
Current Total: 500 | Remaining: 1500  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 4  
  
Enter custom calories: 120  
  
Current Total: 620 | Remaining: 1380  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: |
```

Figure 5.4: Custom Calorie Input Feature

In order to handle those food items that are not included in the preset menu, a "Custom Input" option (Option 4) has been provided in the system. This option permits the user to input a particular integer value. The system will capture this input and perform addition operation with total value of calorie. This shows the flexibility of the assembly program in processing multiple type of data entered by the user.

5.5 Warning Mechanism and Negative Balance

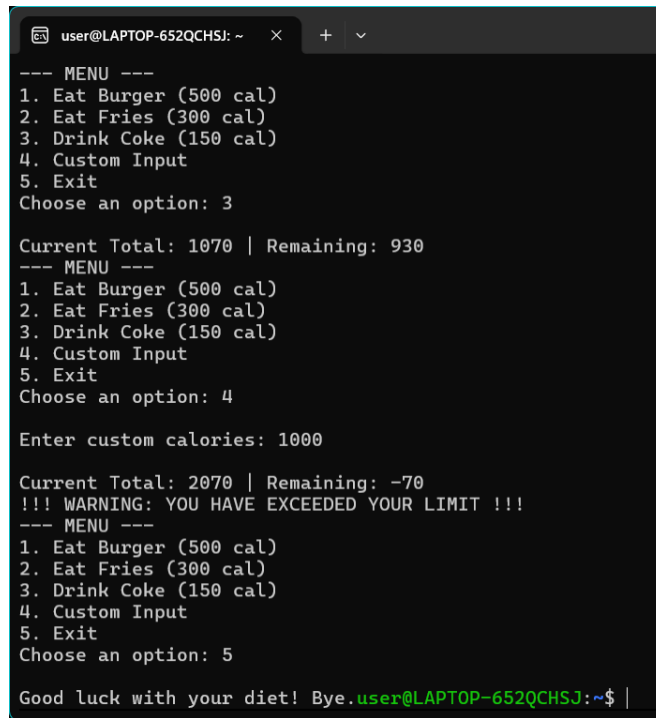
A terminal window titled 'user@LAPTOP-652QCHSJ: ~' showing a calorie tracking application. The application displays the current total and remaining calories, a menu of food items with their calorie counts, and a prompt to choose an option. The user selects option 3 (Drink Coke) and then option 4 (Custom Input). The application then prompts for custom calories (1000) and displays a warning message: '!!! WARNING: YOU HAVE EXCEEDED YOUR LIMIT !!!' because the remaining balance has become negative (-70). The application then displays the menu again and prompts for another option.

```
user@LAPTOP-652QCHSJ: ~  
Current Total: 920 | Remaining: 1080  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 3  
  
Current Total: 1070 | Remaining: 930  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 4  
  
Enter custom calories: 1000  
  
Current Total: 2070 | Remaining: -70  
!!! WARNING: YOU HAVE EXCEEDED YOUR LIMIT !!!  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: |
```

Figure 5.5: Warning Alert and Negative Balance Handling

Another practical feature of the system is the input validation logic. In case the total calorie input go beyond the user's defined limit, the system will perform with conditional jump and display the "WARNING" message. In addition, the system implements the logic of signed integer arithmetic. As shown above, the system will perform the correct calculation and display the remaining negative amount (e.g., -70).

5.6 Program Termination



```
user@LAPTOP-652QCHSJ: ~  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 3  
  
Current Total: 1070 | Remaining: 930  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 4  
  
Enter custom calories: 1000  
  
Current Total: 2070 | Remaining: -70  
!!! WARNING: YOU HAVE EXCEEDED YOUR LIMIT !!!  
--- MENU ---  
1. Eat Burger (500 cal)  
2. Eat Fries (300 cal)  
3. Drink Coke (150 cal)  
4. Custom Input  
5. Exit  
Choose an option: 5  
  
Good luck with your diet! Bye.user@LAPTOP-652QCHSJ:~$ |
```

Figure 5.6: Program Termination

The choice to exit the application is given by choosing Option 5. This gives user the choice to exit the application in a controlled manner. It simply calls the system exit function (`sys_exit`), and a farewell message is displayed before exiting the application.

6.0 Source Code with Explanation

This section offers a clear explanation of the source code used in the system. The source code is divided into two files: functions.asm, which contains the utility function, and main.asm, which contains the main execution logic.

6.1 Utility Functions (functions.asm)

To ensure the usability and efficiency of the project, this functions.asm file contains all of the reusable utility functions which will be called by main execution code. For example, Integer Print(iprint), ASCII to Integer (ATOI), String Length(slen), String Print(sprint), and quit.

6.1.1 Integer to String Conversion (iprint)

```
iprint:
    push    eax
    push    ecx
    push    edx
    push    esi

    mov     ecx, 0

divide_loop:
    inc     ecx
    mov     edx, 0
    mov     esi, 10
    idiv    esi

    add     edx, 48
    push    edx

    cmp     eax, 0
    jnz     divide_loop

print_loop:
    dec     ecx
    mov     eax, esp
    call    sprint
    pop     eax

    cmp     ecx, 0
    jnz     print_loop

    pop     esi
    pop     edx
    pop     ecx
    pop     eax
    ret
```

Figure 6.1.1: Integer Print function (iprint)

The first section of `iprint` function is component from `push eax` to `mov ecx, 0`. All of the pushing operation is to backup the data in registers, to prevent the race condition while `functions.asm` and `main.asm` are using the registers simultaneously. Initialization of `ECX` is done by `mov ecx, 0` as the function needs a clean counter register to count the looping later.

Second section, the `divide_loop` component. In this part, `inc ecx` is used to increase the counter to track the number of digits. Then, `idiv esi` divides the value in `EAX` by 10. The remainder (the last digit) is stored in `EDX`. Since this is a number, 48 is added to it to convert the integer into an ASCII character. After that, `push edx` places this character onto the stack to save it temporarily. The code then checks if `EAX` is 0 using `cmp`, and if not, the loop continues.

Third section, the `print_loop`. To retrieve the numbers, `dec ecx` is used to count down. The register `eax` is pointed to `esp` (the stack pointer) to access the character that was just saved. Then, the system will call `sprint` function to print the saved character to the screen. Finally, `pop eax` removes the character from the stack to keep the stack clean

6.1.2 ASCII to Integer Conversion (atoi)

This function is crucial because the default keyboard input is always a String, but an Integer is required while executing mathematical operations.

```

atoi:
    push    ebx
    push    ecx
    push    edx
    push    esi

    ; -- Initialize Stage --
    mov     esi, eax
    mov     eax, 0
    mov     ecx, 0

.top:
    xor     ebx, ebx           ; [C]
    mov     bl, [esi]         ; [Re]

    ; -- Validation Check -- (ATC
    cmp     bl, 48             ; [CH
    jl      .done              ; [Ju
    cmp     bl, 57             ; [CH
    jg      .done              ; [Ju

    ; -- Mathematical Conversion
    sub     bl, 48             ; [Co

    mov     ecx, eax           ; [Ba
    mov     eax, 10            ; [Pr
    mul     ecx                 ; [Ma

    add     eax, ebx           ; [Ma

    inc     esi                ; [Ne
    jmp     .top               ; [Lo

.done:
    pop     esi
    pop     edx
    pop     ecx
    pop     ebx
    ret

```

Figure 6.1.2: ASCII to Integer function (ATOI)

First, the required register is all initialized to 0 to store the result later. Inside the loop, one character is read at a time into bl. cmp is used to validate the character. If it is smaller than '0' or bigger than '9', the function stops. To perform the conversion, 48 is subtracted from the character. Then, before adding this new digit, the current total in eax is multiplied by 10 using mul. This shifts the number to the left (like changing 5 to 50) to make space for the new digit.

6.1.3 System Call Wrappers (slen, sprint, quit)

These functions are used to communicate directly with the Linux system to print text and stop the program.

```
slen:
    push    ebx
    mov     ebx, eax

.next:
    cmp     byte [eax], 0
    jz      .finished
    inc     eax
    jmp     .next

.finished:
    sub     eax, ebx
    pop     ebx
    ret
```

Figure 6.1.3: System Call Wrappers (slen)

```
sprint:
    push    edx
    push    ecx
    push    ebx
    push    eax

    call    slen

    mov     edx, eax
    pop     eax

    mov     ecx, eax
    mov     ebx, 1
    mov     eax, 4

    int     80h

    pop     ebx
    pop     ecx
    pop     edx
    ret
```

Figure 6.1.3: System Call Wrappers (sprint)

The slen function first assigns the start address to ebx. Then uses eax to loop through the string until a 0 value is found. Lastly, it uses subtraction to subtract the value of start address and the current value of eax address and get the length of string.

The sprint function first prepares the registers for the system call. The length is placed in edx from eax. The system pop back the string address into eax and place it into ecx. 1 in ebx is indicating output to screen, while 4 in eax is indicating system write. Then, int 80h instructs the kernel to execute the command.

6.2 Main Program Initialization (main.asm)

This section defines the variables and handles the program start.

```
%include 'functions.asm'

section .data
; --- Prompt Messages ---
msg_ask_goal      db  'Enter your daily calorie limit (e.g. 2000): ', 0h
msg_status_tot    db  0Ah, 'Current Total: ', 0h ; 0Ah = Newline character
msg_status_rem    db  ' | Remaining: ', 0h
msg_warning       db  0Ah, '!!! WARNING: YOU HAVE EXCEEDED YOUR LIMIT !!!', 0h
msg_minus         db  '-', 0h ; Minus sign for negative numbers

; --- Menu Interface ---
msg_menu_title    db  0Ah, '--- MENU ---', 0h
msg_opt1          db  0Ah, '1. Eat Burger (500 cal)', 0h
msg_opt2          db  0Ah, '2. Eat Fries (300 cal)', 0h
msg_opt3          db  0Ah, '3. Drink Coke (150 cal)', 0h
msg_opt4          db  0Ah, '4. Custom Input', 0h
msg_opt5          db  0Ah, '5. Exit', 0h
msg_prompt        db  0Ah, 'Choose an option: ', 0h
msg_invalid       db  0Ah, 'Invalid choice!', 0h
msg_bye           db  0Ah, 'Good luck with your diet! Bye.', 0h

; --- Custom Input Message ---
msg_ask_custom    db  0Ah, 'Enter custom calories: ', 0h

section .bss
; --- Memory Variables ---
daily_limit       resd 1 ; [Variable] Reserve 4 bytes (Double word) for calorie limit
current_total     resd 1 ; [Variable] Reserve 4 bytes for current intake
input_buffer      resb 255 ; [Buffer] Reserve 255 bytes for user input string

section .text
global _start
```

Figure 6.2: Main Program Initialization

6.2.1 Data and Memory Allocation

In main.asm, the system included the functions.asm to call the defined utility functions later. By the section of .data is to define the static message variable which will be shown to user. In the .bss section, resd 1 is used to reserve 4 bytes of memory for daily_limit and current_total. This serves as storage for the user's data.

6.2.2 Program Entry and Goal Setting

```
_start:
; -----
; 1. Initialization Phase
; -----
mov     dword [current_total], 0

; --- Ask for Daily Goal ---
mov     eax, msg_ask_goal
call    sprint

; --- Read User Input ---
mov     edx, 255
mov     ecx, input_buffer
mov     ebx, 0
mov     eax, 3
int     80h

; --- Convert and Store ---
mov     eax, input_buffer
call    atoi
mov     [daily_limit], eax
```

Figure 6.2.2: Program Entry and Goal Setting

Inside `_start`, the system first prompts the user for their goal. `sys_read (eax=3)` is used to capture input. Subsequently, the `atoi` function is called to convert the input string into a number, which is then saved into the `[daily_limit]` variable using `mov`.

6.3 The Main Application Loop (main.asm)

The menu_loop is the core component that runs continuously to refresh the screen.

6.3.1 Status Display and Signed Arithmetic Logic

```
menu_loop:
; --- A. Display Current Total
mov     eax, msg_status_tot
call    sprint

mov     eax, [current_total]
call    iprint

; --- B. Calculate and Display
mov     eax, msg_status_rem
call    sprint

; [Calculation] Remaining = Li
mov     eax, [daily_limit]
sub     eax, [current_total]

; [Check] Is the result negati
cmp     eax, 0
jge     print_positive

; [Negative Handler] If negati
push    eax
mov     eax, msg_minus
call    sprint
pop     eax
neg     eax
```

Figure 6.3.1: Status Display and Signed Arithmetic Logic

First, current_total is printed. Then, the remaining balance is calculated using the sub instruction: daily_limit minus current_total. A special check is implemented here. The result is compared with 0. If it is positive, it is printed normally. Handling Negative Numbers: If the result is negative (indicating the user exceeded the limit), the iprint function cannot handle it directly. Therefore, a " - " sign is manually printed first. Then, the neg instruction is used. This flips the negative number to positive (e.g., -500 to 500), allowing it to be printed correctly.

6.3.2 Warning Logic

```
print_positive:
    call    iprint                ; [Print] Print the (now positive) remaining value

    ; --- C. Warning System ---
    ; Check if Total > Limit to trigger warning message
    mov     eax, [current_total]
    cmp     eax, [daily_limit]
    jle     show_menu            ; [Jump] If Total <= Limit, skip warning

    ; [Warning] Display warning if limit exceeded
    mov     eax, msg_warning
    call    sprint
```

Figure 6.3.2: Warning Logic

mov is used to load current_total, and cmp compares it against daily_limit. If the total is greater (jg), the program jumps to the warning section to print an alert. Otherwise, it skips this part and proceeds to show the menu.

6.4 Menu Selection and Execution(main.asm)

6.4.1 Menu Display and Input

```
show_menu:
; --- D. Display Menu Options ---
mov     eax, msg_menu_title
call    sprint
mov     eax, msg_opt1
call    sprint
mov     eax, msg_opt2
call    sprint
mov     eax, msg_opt3
call    sprint
mov     eax, msg_opt4
call    sprint
mov     eax, msg_opt5
call    sprint

mov     eax, msg_prompt
call    sprint

; --- E. Read User Choice ---
mov     edx, 255
mov     ecx, input_buffer
mov     ebx, 0
mov     eax, 3
int     80h
```

Figure 6.4.1: Menu Display and Input

The menu title and all options (1 to 5) are printed to the screen using the sprint function. Then, the program waits for the user to type a choice using sys_read.

6.4.2 Jump Table Implementation

```
; --- F. Input Dispatcher (Switch)
mov     esi, input_buffer

cmp     byte [esi], '1'
je      add_burger

cmp     byte [esi], '2'
je      add_fries

cmp     byte [esi], '3'
je      add_coke

cmp     byte [esi], '4'
je      add_custom

cmp     byte [esi], '5'
je      exit_program

; [Error] Handle invalid input
mov     eax, msg_invalid
call    sprint
jmp     menu_loop
```

Figure 6.4.2: *Jump Table Implementation*

Once the user has typed a choice, only the first byte of the typed string is checked. A series of `cmp` instructions are used to check if the typed string is '1', '2', '3', '4', or '5'. If the typed string is '1', the `je` (Jump Equal) instruction is used to jump to the `add_burger` section. If the typed string is '5', the program jumps to `exit`. Otherwise, it jumps back to the start of the loop.

6.4.3 Feature Execution (Standard vs. Custom)

```
add_burger:
    mov     eax, [current_total]
    add     eax, 500
    mov     [current_total], eax
    jmp     menu_loop

add_fries:
    mov     eax, [current_total]
    add     eax, 300
    mov     [current_total], eax
    jmp     menu_loop

add_coke:
    mov     eax, [current_total]
    add     eax, 150
    mov     [current_total], eax
    jmp     menu_loop

add_custom:
    ; --- Step 1: Ask for Input ---
    mov     eax, msg_ask_custom
    call    sprint

    ; --- Step 2: Read Input ---
    mov     edx, 255
    mov     ecx, input_buffer
    mov     ebx, 0
    mov     eax, 3
    int     80h

    ; --- Step 3: Convert to Integer ---
    mov     eax, input_buffer
    call    atoi

    ; --- Step 4: Add to Total ---
    add     [current_total], eax
    jmp     menu_loop
```

Figure 6.4.3: Feature Execution (Standard & Custom)

For the standard items like the Burger, the add instruction is used to add 500 to the [current_total] variable. For Option 4 Custom Input, the same code is used again. The user is prompted to type a number, and then the new value is added to the total. This makes the system more flexible.

6.4.4 Exit Program

```
exit_program:  
    mov     eax, msg_bye  
    call    sprint  
    call    quit
```

Figure 6.4.4: Exit program function

Once option 5 has been typed, the system prints a goodbye message. Then, it calls the quit function to inform the operating system that the program wants to stop.

7.0 Conclusion

In this assignment, a basic system, "Daily Calorie Budget Management System" was successfully created using the x86 assembly language. The system was able to fulfill all the requirements as set out in the module guidelines, which include the implementation of arithmetic operations, iterative loops, as well as conditional branching statements (jump).

During the development of this project, extensive knowledge was gained regarding the underlying operations of computer architecture. Some of the technical skills learned which include the utilization of the registers EAX, EBX, ECX, as well as EDI for the storage of temporary data, and the use of the Stack for complex data conversion procedures such as ITOA and ATOI. Moreover, the ability to deal with signed integers was also learned, thus the system is able to display negative balances rather than an overflow error when the user exceeds their calorie limit.

However, the system has some limitations. First, the current menu input logic only reads the first character of the input buffer for efficiency. This is problematic because, although it works for general cases, it does not strictly validate the characters at the end of the input stream. Second, the current system stores its data in volatile memory, meaning that all its progress is lost after the program ends.

As for further improvements, a "File I/O" feature could be added into the system, which would allow the user to save their daily records in a file. This would be very useful for long-term tracking. Another improvement could be to add a new input validation logic that would flush the input buffer for a smoother user experience. To conclude, this project has given us a good introduction on how high-level software can interact with hardware components directly in the architecture level of understanding.

8.0 References

Irvine, K. R. (2020). *Assembly language for x86 processors* (8th ed.). Pearson.

Stallings, W. (2022). *Computer organization and architecture: Designing for performance* (11th ed.). Pearson.

Attari, M. (2023). *Importance of Assembly Language in Cyber Security and Reverse Engineering*. ResearchGate.

EC-Council. (2022). *Malware Reverse Engineering for Beginners Explained*. EC-Council Blog.

Cobalt. (2023). *A Pentester's Guide to Exploiting Buffer Overflow Vulnerabilities*. Cobalt.io.

Appendices

Complete Source Code

main.asm

```

GNU nano 7.2 main.asm
=====
File Name: main.asm
Purpose: Daily Calorie Budget Manager (Main Logic)
Language: x86 Assembly (NASM)
Description: Handles user input, arithmetic operations, loop control,
            and conditional logic for calorie tracking.
=====
%include 'functions.asm'

section .data
; --- Prompt Messages ---
msg_ask_goal    db 'Enter your daily calorie limit (e.g. 2000): ', 0h
msg_status_tot  db 0Ah, 'Current Total: ', 0h ; 0Ah = Newline character
msg_status_rem  db ' | Remaining: ', 0h
msg_warning     db 0Ah, '!!! WARNING: YOU HAVE EXCEEDED YOUR LIMIT !!!', 0h
msg_minus       db '-', 0h ; Minus sign for negative numbers

; --- Menu Interface ---
msg_menu_title  db 0Ah, '--- MENU ---', 0h
msg_opt1        db 0Ah, '1. Eat Burger (500 cal)', 0h
msg_opt2        db 0Ah, '2. Eat Fries (300 cal)', 0h
msg_opt3        db 0Ah, '3. Drink Coke (150 cal)', 0h
msg_opt4        db 0Ah, '4. Custom Input', 0h
msg_opt5        db 0Ah, '5. Exit', 0h
msg_prompt      db 0Ah, 'Choose an option: ', 0h
msg_invalid     db 0Ah, 'Invalid choice!', 0h
msg_bye         db 0Ah, 'Good luck with your diet! Bye.', 0h

; --- Custom Input Message ---
msg_ask_custom  db 0Ah, 'Enter custom calories: ', 0h

section .bss
; --- Memory Variables ---
daily_limit     resd 1 ; [Variable] Reserve 4 bytes (Double word) for calorie limit
current_total    resd 1 ; [Variable] Reserve 4 bytes for current intake
input_buffer     resb 255 ; [Buffer] Reserve 255 bytes for user input string

section .text
global _start

_start:
;
; 1. Initialization Phase
;
mov     dword [current_total], 0 ; [Init] Ensure current total starts at 0

; --- Ask for Daily Goal ---
mov     eax, msg_ask_goal ; [Prompt] Load address of prompt message
call    sprint ; [Print] Print message to screen

; --- Read User Input ---
mov     ecx, 255 ; [Arg 3] Max length to read
mov     ecx, input_buffer ; [Arg 2] Store input in 'input_buffer'
mov     ebx, 0 ; [Arg 1] File Descriptor 0 = Stdin (Keyboard)
mov     eax, 3 ; [Syscall] System Call 3 = sys_read
int     80h ; [Interrupt] Execute read

; --- Convert and Store ---
mov     eax, input_buffer ; [Arg] Load address of input string
call    atoi ; [Convert] Convert string to integer (Result in EAX)
mov     [daily_limit], eax ; [Store] Save the integer value into 'daily_limit' variable

;
; 2. Main Application Loop
;
menu_loop:
; --- A. Display Current Total ---
mov     eax, msg_status_tot ; [Prompt] "Current Total: "
call    sprint

mov     eax, [current_total] ; [Load] Load value from memory variable
call    iprint ; [Print] Print integer to screen

; --- B. Calculate and Display Remaining ---
mov     eax, msg_status_rem ; [Prompt] " | Remaining: "
call    sprint

; [Calculation] Remaining = Limit - Current Total
mov     eax, [daily_limit] ; Load Limit
sub     eax, [current_total] ; Subtract Total from Limit

```

```

; [Check] Is the result negative? (Signed Comparison)
cmp     eax, 0
jge     print_positive      ; [Jump] If EAX >= 0, jump to normal printing

; [Negative Handler] If negative, print minus sign and convert to positive
push    eax                 ; [Backup] Save the negative value
mov     eax, msg_minus      ; [Print] Print "-" character
call    sprint
pop     eax                 ; [Restore] Restore value
neg     eax                 ; [Math] Negate EAX (e.g., -500 becomes 500) for printing

print_positive:
call    iprint              ; [Print] Print the (now positive) remaining value

; --- C. Warning System ---
; Check if Total > Limit to trigger warning message
mov     eax, [current_total]
cmp     eax, [daily_limit]
jle     show_menu          ; [Jump] If Total <= Limit, skip warning

; [Warning] Display warning if limit exceeded
mov     eax, msg_warning
call    sprint

show_menu:
; --- D. Display Menu Options ---
mov     eax, msg_menu_title
call    sprint
mov     eax, msg_opt1
call    sprint
mov     eax, msg_opt2
call    sprint
mov     eax, msg_opt3
call    sprint
mov     eax, msg_opt4
call    sprint
mov     eax, msg_opt5
call    sprint

mov     eax, msg_prompt     ; [Prompt] "Choose an option: "
call    sprint

; --- E. Read User Choice ---
mov     edx, 255            ; [Arg] Buffer size
mov     ecx, input_buffer   ; [Arg] Input buffer
mov     ebx, 0              ; [Arg] Stdin
mov     eax, 3              ; [Syscall] sys_read
int     80h

; --- F. Input Dispatcher (Switch Case) ---
mov     esi, input_buffer   ; Point to input

cmp     byte [esi], '1'     ; [Check] Is input '1'?
je      add_burger         ; [Jump] Go to Burger logic

cmp     byte [esi], '2'     ; [Check] Is input '2'?
je      add_fries          ; [Jump] Go to Fries logic

cmp     byte [esi], '3'     ; [Check] Is input '3'?
je      add_coke           ; [Jump] Go to Coke logic

cmp     byte [esi], '4'     ; [Check] Is input '4'?
je      add_custom         ; [Jump] Go to Custom Input logic

cmp     byte [esi], '5'     ; [Check] Is input '5'?
je      exit_program       ; [Jump] Go to Exit logic

; [Error] Handle invalid input
mov     eax, msg_invalid
call    sprint
jmp     menu_loop          ; [Loop] Return to start of loop

; -----
; 3. Action Handlers
; -----

add_burger:
mov     eax, [current_total] ; [Load] Get current total
add     eax, 500             ; [Math] Add 500 calories
mov     [current_total], eax ; [Store] Save new total
jmp     menu_loop           ; [Loop] Return to menu

add_fries:
mov     eax, [current_total]
add     eax, 300             ; [Math] Add 300 calories
mov     [current_total], eax
jmp     menu_loop

add_coke:
mov     eax, [current_total]
add     eax, 150             ; [Math] Add 150 calories
mov     [current_total], eax
jmp     menu_loop

add_custom:
; --- Step 1: Ask for Input ---
mov     eax, msg_ask_custom
call    sprint

; --- Step 2: Read Input ---
mov     edx, 255
mov     ecx, input_buffer
mov     ebx, 0
mov     eax, 3
int     80h

; --- Step 3: Convert to Integer ---
mov     eax, input_buffer
call    atoi                 ; [Convert] EAX now holds the custom number

; --- Step 4: Add to Total ---
add     [current_total], eax ; [Math] Direct memory addition: Total = Total + EAX
jmp     menu_loop           ; [Loop] Return to menu

exit_program:
mov     eax, msg_bye
call    sprint
call    quit                 ; [Exit] Terminate program

```

functions.asm

```

GNU nano 7.2                                     functions.asm
;-----
; FUNCTION: iprint (Integer Print)
; Purpose: Converts an integer in EAX to a string and prints it to stdout.
; Algorithm: Repeatedly divide by 10, push remainders to stack, then pop and print.
;-----
iprint:
    push    eax                ; [Backup] Save registers in stack to preserve main program state
    push    ecx
    push    edx
    push    esi

    mov     ecx, 0             ; [Counter] Initialize digit counter ECX to 0

divide_loop:
    inc     ecx                ; [Count] Increment digit counter (we found a new digit), ecx = ecx + 1
    mov     edx, 0             ; [Clean] Clear EDX because DIV uses EDX:EAX
    mov     esi, 10            ; [Divisor] Set divisor to 10
    idiv    esi                ; [Divide] EAX = EAX / 10, EDX = Remainder (the digit)

    add     edx, 48             ; [Convert] Convert int (0-9) to ASCII ('0'-'9') by adding 48
    push    edx                ; [Save] Push the ASCII character onto the Stack (LIFO)

    cmp     eax, 0             ; [Check] Is the quotient 0? (Are we done?)
    jnz     divide_loop        ; [Loop] If not 0, jump back to divide the rest

print_loop:
    dec     ecx                ; [Countdown] Decrease the digit counter
    mov     eax, esp            ; [Extended Stack Pointer] Point EAX to the character currently on top of Stack
    call    sprint              ; [Print] Call our string print function
    pop     eax                ; [Remove] Pop the character from Stack to clean up

    cmp     ecx, 0             ; [Check] Have we printed all digits?
    jnz     print_loop         ; [Loop] If not, continue printing

    pop     esi                ; [Restore] Restore original register values
    pop     edx
    pop     ecx
    pop     eax
    ret                        ; [Return] Go back to main program

;-----
; FUNCTION: atoi (ASCII to Integer)
; Purpose: Converts a string pointer in EAX to an integer value in EAX.
; Logic: Iterates through string, multiplies current total by 10 and adds new digit.
;-----
atoi:
    push    ebx                ; [Backup] Save registers to preserve state
    push    ecx
    push    edx
    push    esi

    ; -- Initialize Stage --
    mov     esi, eax            ; [Pointer] Move string address to ESI for reading
    mov     eax, 0              ; [Result] Initialize result accumulator to 0
    mov     ecx, 0              ; [Temp] Initialize temporary register for calculation later

.top:
    xor     ebx, ebx            ; [Clean] Clear EBX to ensure we read a clean byte
    mov     bl, [esi]           ; [Read] Load a single byte (character) into BL

    ; -- Validation Check -- (ATOI - integer only accept 0-9)
    cmp     bl, 48              ; [Check] Is character < '0' (ASCII 48)?
    jl      .done               ; [Jump] If yes (e.g., Newline or Space), stop conversion
    cmp     bl, 57              ; [Check] Is character > '9' (ASCII 57)?
    jg      .done               ; [Jump] If yes, stop conversion

    ; -- Mathematical Conversion --
    sub     bl, 48              ; [Convert] Convert ASCII to Integer, BL = BL - 48

    mov     ecx, eax            ; [Backup] Store current total in ECX
    mov     eax, 10             ; [Prepare] Set multiplier to 10
    mul     ecx                 ; [Math] EAX = EAX * 10 (Shift decimal place left)

    add     eax, ebx            ; [Math] EAX = (Old_Total * 10) + New_Digit

    inc     esi                 ; [Next] Move pointer to the next character
    jmp     .top                ; [Loop] Jump back to process the next digit

.done:
    pop     esi                ; [Restore] Restore registers in reverse order
    pop     edx
    pop     ecx
    pop     ebx
    ret                        ; [Return] Final integer is now in EAX

;-----
; FUNCTION: slen (String Length)
; Purpose: Calculates the length of the string pointed to by EAX.
; Returns: Length in EAX.
;-----
slen:
    push    ebx                ; [Backup] Save EBX to preserve state
    mov     ebx, eax            ; [Save Start] Copy starting address to EBX

.next:
    cmp     byte [eax], 0        ; [Check] Is the current character NULL (0)?
    jz      .finished           ; [Jump] If Zero, we reached the end of the string
    inc     eax                 ; [Next] Move pointer to the next character
    jmp     .next                ; [Loop] Repeat check

.finished:
    sub     eax, ebx            ; [Calc] Length = Current Address (EAX) - Start Address (EBX)
    pop     ebx                ; [Restore] Restore EBX original value
    ret                        ; [Return] Length is now in EAX

```

```

-----
FUNCTION: sprint (String Print)
Purpose: Prints the string in EAX to stdout via System Call.
-----
sprint:
    push    edx            ; [Backup] Save EDX
    push    ecx            ; [Backup] Save ECX
    push    ebx            ; [Backup] Save EBX
    push    eax            ; [Backup] Save String Address (Important!)

    call    slen           ; [Calc] Get string length (Result is in EAX)

    mov     edx, eax       ; [Arg 3] Move Length to EDX (Required for sys_write)
    pop     eax            ; [Restore] Retrieve String Address from Stack into EAX

    mov     ecx, eax       ; [Arg 2] Move String Address to ECX (Required for sys_write)
    mov     ebx, 1         ; [Arg 1] File Descriptor 1 = Standard Output (Screen)
    mov     eax, 4         ; [Syscall] System Call Number 4 = sys_write

    int     80h           ; [Interrupt] Call Linux Kernel to execute the write

    pop     ebx            ; [Restore] Restore EBX
    pop     ecx            ; [Restore] Restore ECX
    pop     edx            ; [Restore] Restore EDX
    ret                     ; [Return]

-----
FUNCTION: quit (Exit Program)
Purpose: Terminates the program cleanly using the sys_exit system call.
-----
quit:
    mov     ebx, 0         ; [Status] Load exit status 0 (Indicates Success/No Error)
    mov     eax, 1         ; [Syscall] Load system call number 1 (sys_exit)
    int     80h           ; [Interrupt] Call Kernel to terminate the process
    ret                     ; [Unreachable] This instruction is never reached as process ends

```